Alani - Executive Summary

This report outlines a complete specification and delivery plan for **Alani**, a bare-metal "cognitive kernel" operating system implemented in Rust. We propose splitting Alani into multiple Git repositories (for kernel, runtime, libraries, cognition engine, memory, devices, security, audit, terminal, and userspace) with a unified workspace for building and testing. The solution includes 62 specification documents (with titles, purposes, audiences, sections, lengths, and inter-document dependencies), example repository templates (directory trees, Cargo projects, CI configs), a plan to build a labeled data corpus, and detailed development workflow and packaging procedures. We assume a target platform of 64-bit x86 (or RISC-V) hardware, an open-source license, and Rust 2021 edition using stable toolchains. Figures below illustrate typical OS architectures and the proposed repository dependency graph. All code and docs follow official Rust guidelines[1][2].

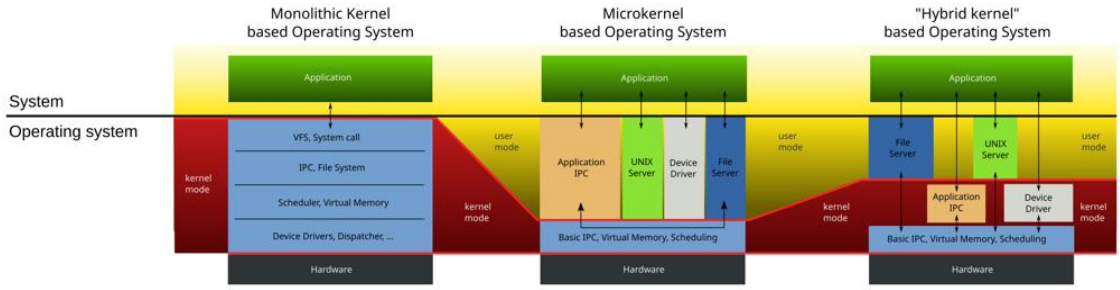

Figure: Common OS architectural styles. Alani adopts a hybrid design (kernel services plus user-space subsystems) combining elements of monolithic and microkernel architectures[3][4].

1. Document Specification

We define 62 documents covering Alani's design, usage, development, and dataset. Each document has a title, purpose, audience, structure, estimated length, and dependencies on other docs. The table below summarizes the set.

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
1	**Alani Overview**	High-level project summary, goals, scope.	Project sponsors, architects, developers	Abstract, goals, scope, terms, references	5 pages	–
2	**System Model**	Overall system model (hardware assumptions,	Architects, dev leads	Hardware, OS layers, execution modes, primitives	5 pp	Doc1 (Overview)

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
		execution model).				
3	**Design Principles**	Core principles (modularity, safety, cognition, no_std, etc.).	Architects, designers	Safety, concurrency, determinism, cognitive abstractions	3 pp	Doc2
4	**Use Cases & Requirements**	Key use cases (AI agents, real-time control) and high-level requirements (functional, performance, security, etc.).	Stakeholders, devs	Use-case scenarios, functional reqs, non-functional reqs	8 pp	Doc1, Doc2
5	**Architecture Overview**	Big-picture architecture diagram and description.	Architects, new devs	Component diagram, data/control flows, layers	6 pp	Doc2, Doc3
6	**Kernel Architecture**	Kernel subsystems (scheduler, memory manager, IPC, etc.).	Kernel devs, systems engineers	Kernel structure, boot, init sequence, module interfaces	10 pp	Doc5
7	**Cognitive Subsystem**	Design of the AI reasoning core (neural engines, knowledge store).	Cognition devs	AI service architecture, models, knowledge APIs	8 pp	Doc5
8	**Syscall Interface**	Definition of system calls for cognitive tasks.	Kernel & runtime devs	Syscall table, call conventions, data types, error model	6 pp	Doc6, Doc7
9	**Syscall Reference**	Detailed syscall API (like `sys_mem_allocate`, `sys_device_call`).	Devs (kernel & user)	Each call: purpose, params, return values, errors	10 pp	Doc8

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
10	**Kernel ABI (Rust)**	Rust definitions of kernel–user ABI (structs, traits).	Devs, FFI authors	Rust code listings (traits, types), binary interface	5 pp	Doc9
11	**Memory Management**	Physical/virtual memory model (paging, allocators, persistence).	Kernel devs	VM setup, alloc schemes, buffer management	7 pp	Doc6
12	**Scheduling & Multitasking**	Task scheduling policies (real-time, fairness, cognitive priorities).	Kernel devs	Scheduler design, algorithms, priority scheme	6 pp	Doc6
13	**Device Model**	Device driver framework (interrupts, DMA, device abstractions).	Kernel/devs	Driver API, lifecycle, registration, polling vs IRQ	6 pp	Doc6
14	**Cognitive Devices**	"Model devices" and "memory devices" (AI accelerators, knowledge stores).	Cog subsystem devs	Device roles, data flow with devices, APIs	6 pp	Doc7, Doc13
15	**Security Model**	Threat model, security goals (isolation, attestation, etc.).	Security architects	Principals, trust boundaries, confidentiality/integrity	6 pp	Doc4 (requirements)
16	**Security Mechanisms**	Authentication, authorization, encryption, sandboxing, capabilities.	Security devs	Access control, crypto APIs, secure boot	6 pp	Doc15
17	**Observability & Tracing**	Logging, telemetry, metrics design.	Sysadmins, devs	Trace formats, event log schema, monitoring interfaces	5 pp	Doc6, Doc7, Doc16

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
18	**Auditing & Forensics**	Audit framework (tamper-evident logs, audit policy).	Security auditors	Audit rules, log integrity, review procedures	5 pp	Doc16, Doc17
19	**Minimal Viable Kernel (MVK)**	Requirements for a proof-of-concept kernel.	Dev leads	MVK scope, prioritized features, deliverables	3 pp	Doc4, Doc6
20	**Differentiation**	How Alani differs from other OS/AI platforms.	Stakeholders	Key novel points, citations, benchmarks	4 pp	Doc4, Doc5
21	**Future Work**	Planned features beyond MVK (extended cognition, networking, etc.).	Planners, devs	Feature roadmap, research directions	4 pp	Doc19, Doc20
22	**Glossary & Acronyms**	Definitions of terms used in all docs.	All readers	Terms A–Z, acronyms list	3 pp	–
23	**alani-kernel: README**	Repo README for kernel (setup, build, run instructions).	Kernel devs, integrators	Overview, build/run steps, repo structure	2 pp	Doc1, Doc6
24	**alani-kernel: Design Doc**	Detailed design of kernel repo (maps to docs 6–14).	Kernel devs	Link to spec, coding guidelines for kernel	3 pp	Doc6–Doc14
25	**alani-runtime: README**	Repo README for runtime system (user-space executor).	Runtime devs	Overview, usage, dependencies	2 pp	Doc6, Doc8
26	**alani-runtime: Design**	Design of runtime environment	Runtime devs	Lifecycle, integration with kernel, syscalls	3 pp	Doc8, Doc11

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
		(task loader, process model).				
27	**alani-lib: README**	Core library README (shared utilities).	Devs	Usage examples, API summary	1 pp	–
28	**alani-lib: API**	Public Rust API spec for alani-lib (helper functions, types).	Devs	Functions/modules list with docs	3 pp	–
29	**alani-cognition: README**	Cognitive engine README (model loader, inference services).	AI devs	Overview, dependencies (TensorFlow/Rust?), usage	2 pp	Doc7
30	**alani-cognition: Design**	Details of cognition engine (data pipelines, learning).	AI devs	Architecture diagrams, key algorithms	4 pp	Doc7, Doc14
31	**alani-memory: README**	Memory repo README (persistent memory simulation).	Memory devs	Overview, building, running	2 pp	Doc11
32	**alani-memory: API**	Public API of memory subsystem (interfaces to NVM, caching).	Memory devs	Structs, traits for memory devices	3 pp	Doc11
33	**alani-devices: README**	Devices repo README (I/O and peripheral support).	Driver devs	Overview, device classes	2 pp	Doc13
34	**alani-devices: API**	Device driver API spec (trait definitions).	Driver devs	`Driver` traits, helper macros	3 pp	Doc13
3	**alani-**	Security repo	Security	Overview,	2 pp	Doc16

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
5	**security: README**	README (authentication, crypto libraries).	devs	dependencies (ring, sodiumoxide, etc.)		
36	**alani-security: API**	Security primitives API (auth, crypto).	Security devs	Public functions/types	3 pp	Doc16
37	**alani-audit: README**	Audit repo README (logging framework).	Security/Audit devs	Overview, integration with kernel	2 pp	Doc17
38	**alani-audit: API**	Audit APIs (log interfaces, queries).	Audit devs	Classes, persistence format specs	3 pp	Doc17
39	**alani-terminal: README**	Terminal repo README (shell or CLI).	Userspace devs	Overview, usage (commands supported)	2 pp	Doc17
40	**alani-terminal: API**	Terminal application API (if exposed as lib).	Userspace devs	Commands, scripting interface	2 pp	–
41	**alani-userspace: README**	Example userspace app/libraries README (e.g. agents CLI).	Userspace devs	Overview, example usage	2 pp	Doc5, Doc19
42	**Inter-Repo Interfaces**	How crates/repos interact (Cargo features, versioning).	Devs, integrators	Dependency graph, version policies	4 pp	Doc6–Doc40
43	**APIs & Data Formats**	Shared data schemas (config, messages, storage formats).	Devs, data teams	JSON/MsgPack schemas, TLV etc., protocol buffers	5 pp	Doc8, Doc14, Doc17
4	**Performan**	Benchmark	Performan	Benchmark	4 pp	Doc12,

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
4	**ce Benchmarking**	design (use Criterion, test scenarios).	ce engineers	cases, metrics, environment specs		Doc13, Doc14
45	**Testing Strategy**	Unit, integration, and end-to-end tests plan.	QA & devs	Testing frameworks, test coverage, CI integration	3 pp	Doc42, Doc44
46	**Quality Assurance**	QA procedures (code coverage, linting, fuzzing, static analysis).	QA & devs	Tools (clippy, MIRI, cargo-audit), metrics targets	3 pp	Doc45
47	**Development Workflow**	Git strategy (branches, PR reviews, issue flow).	All devs	Branch naming, PR policy, issue tagging	5 pp	Doc48–Doc49
48	**Branching & Releases**	Branch management and release tagging.	All devs	GitFlow/GitHub Flow, versioning, changelog process	4 pp	Doc47
49	**Code Review Guidelines**	PR review checklist and guidelines.	All devs	Review steps, approval criteria, required checks	2 pp	Doc47
50	**Continuous Integration**	CI/CD design (build/test pipeline, actions).	DevOps, devs	Workflow files, caching, matrix builds, status badges	4 pp	Doc45, Doc47
51	**Release Packaging**	Packaging binaries, docs, corpus into distributable ZIP.	DevOps, releasers	Script (Cargo package, artifacts, zip layout)	3 pp	Doc50, Doc52
52	**Corpus Creation Plan**	Overall plan for building the Alani corpus.	Data scientists	Steps overview, responsibilities, tools	4 pp	Doc45, Doc50
53	**Data Sources**	Potential datasets (open corpora, logs, knowledge	Data engineers	Data repositories, licensing, data rights	3 pp	Doc52

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
		bases).				
54	**Data Schema & Formats**	Schema definitions for corpus (e.g. JSON fields, schemas).	Data architects	Field definitions, file formats (JSON, CSV, Parquet)	4 pp	Doc53
55	**Labeling Guidelines**	How to annotate or label data (taxonomy, instructions).	Labelers, analysts	Annotation spec, tag definitions, quality control	5 pp	Doc52
56	**Storage & Access Control**	Corpus storage design, DB vs file, access policies.	DevOps, security	Storage platform (S3/DB), ACL, encryption, backups	3 pp	Doc54, Doc55
57	**Sample Data**	Example data files illustrating formats (JSON snippets).	All devs	Sample entries, schema comments	2 pp	Doc54, Doc55
58	**Data Generation Scripts**	Scripts/tools for importing/generating corpus.	DevOps, data eng	Script descriptions (Rust/Python), usage examples	3 pp	Doc57
59	**Doc Templates**	Templates for writing design docs, specs, comments.	All doc authors	Doc structure template, code-comment guidelines	2 pp	–
60	**Code Style Guide**	Rust coding standards (formatting, naming, idioms).	All devs	rustfmt style, clippy lints, module organization	3 pp	–
61	**Contribution Guidelines**	How to contribute (CLA, PR process, CoC).	Contributors	Steps to contribute, review policy, code of conduct	2 pp	Doc47, Doc49
62	**License & Legal**	Project license and legal notes.	All users	License text (MIT/Apache), warranty,	1 pp	–

#	Title	Purpose & Content	Audience	Key Sections (suggested)	Est. Length	Depends On
				trademark notes		

Each document will follow a clear template (title page, overview, sections, references). This table can serve as the *master index* for the Alani spec. Dependencies indicate which docs should be read first (e.g. kernel docs build on the Architecture and Syscall docs).

2. Repository Layouts and APIs

Each Alani repository is a Rust package (crate) or workspace with its own `Cargo.toml`. We recommend using a top-level Cargo *workspace* to coordinate all crates: **Cargo workspaces** let related packages share a single Cargo.lock and `target/` directory[5]. Below are proposed layouts and APIs for each repo. (All repos use Rust 2021 edition and follow official naming/naming conventions[1].)

Mermaid Diagram of repo relationships: The graph below shows how the major crates depend on each other. Edges indicate "uses" or calls into, e.g., the kernel crate's interfaces.

```
graph LR
    alani-kernel --> alani-runtime
    alani-kernel --> alani-memory
    alani-kernel --> alani-devices
    alani-kernel --> alani-security
    alani-kernel --> alani-audit
    alani-runtime --> alani-lib
    alani-cognition --> alani-memory
    alani-cognition --> alani-devices
    alani-terminal --> alani-kernel
    alani-userspace --> alani-runtime
```

alani-kernel

- **Structure:**

```
alani-kernel/
├── Cargo.toml           # package = "alani-kernel", no_std,
dependencies (bootloader, cpu crates, etc.)
├── src/
│   ├── main.rs         # binary entry: kernel init and main loop
│   ├── memory.rs       # memory manager
│   ├── cpu.rs          # CPU/init code (interrupt handlers)
│   ├── scheduler.rs    # task scheduler
│   ├── devices.rs      # device manager
│   ├── syscall.rs      # syscall dispatch
```

```
│   └── security.rs     # privilege checks, encryption utils
├── tests/             # integration tests (simulation)
├── examples/          # simple kernel modules or demos
└── .github/
    └── workflows/ci.yml  # CI pipeline
```

- **Key Crate/Module:** The crate `alani-kernel` (no_std, bare-metal) implements the core OS. It defines public structs/traits for device drivers (`trait KernelDevice`), memory allocators, and exposes syscall numbers (per `syscalls.md`).
- **Public API:** For user-space, the kernel offers a syscall interface via an ABI (see Doc8–Doc10). Internal APIs (for other Rust crates) are kept minimal.
- **Inter-repo interfaces:** No direct Rust dependency on other Alani crates (kernel is final binary). Communication is via syscalls and ABI.
- **CI/CD:** `.github/workflows/ci.yml` runs `cargo build --workspace --verbose` for the kernel target (using e.g. Ghostfrog's rust-os template). It also runs `cargo test` on host (e.g. unit tests with mocks) and `cargo audit` for CVE checks[6].
- **Example `Cargo.toml`:**

```
[package]
name = "alani-kernel"
version = "0.1.0"
edition = "2021"

[dependencies]
bootloader = "0.10"
x86_64 = "0.14"
# ... other low-level crates
```

- **Example `src/main.rs`:**

```
#![no_std]
#![no_main]
use core::panic::PanicInfo;

#[no_mangle]
pub extern "C" fn _start() -> ! {
    // Initialize memory, interrupts, etc.
    init_memory();
    init_scheduler();
    init_devices();
    loop {
        // Kernel main loop
    }
}

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
```

```
    loop {}
}
```

alani-runtime

- **Structure:**

```
alani-runtime/
├── Cargo.toml          # package = "alani-runtime"
├── src/
│   ├── lib.rs          # common code, e.g. task loader
│   └── main.rs         # binary to launch as init (if any)
├── examples/          # e.g. sample user process in Rust
└── .github/workflows/ci.yml
```

- **Purpose:** Provides the user-space runtime environment (perhaps a small process loader or agent dispatcher). It depends on `alani-lib` for common utilities.
- **Public API:** Exposes functions to spawn agents or WASM modules. Uses syscalls to interact with kernel (e.g. `unsafe fn syscall_halt()`).
- **CI/CD:** Builds with `cargo build`. Uses `rustfmt` and `clippy` to enforce style. Tests include calling into a mock kernel API.

alani-lib

- **Structure:**

```
alani-lib/
├── Cargo.toml          # library crate with no_std support
├── src/
│   ├── lib.rs          # re-exports and shared types
│   ├── syscall.rs      # safe wrappers for kernel syscalls
│   ├── utils.rs        # common utilities
└── .github/workflows/ci.yml
```

- **Purpose:** A shared library of core utilities (data types, helpers). For example, an agent can `use alani_lib::SyscallClient;`.
- **Public API:** Traits like `KernelApi` for writing userspace code.
- **CI/CD:** Build + test, `rustdoc` generation.

alani-cognition

- **Structure:**

```
alani-cognition/
├── Cargo.toml          # AI engine crate (CPU and memory models)
├── src/
│   ├── lib.rs          # cognitive core API
│   ├── neural.rs       # neural network routines
```

```
│   ├── memory.rs       # knowledge storage management
└── .github/workflows/ci.yml
```

- **Purpose:** Implements the "cognition" kernel extensions (e.g. loading inference models, managing knowledge base).
- **Public API:** Offers functions like `load_model()`, `run_inference()`, and memory caching. It calls into `alani-memory` and `alani-devices`.
- **CI/CD:** Standard build/test. Benchmarks (Criterion) for key inference kernels (See Doc44[7]).

alani-memory

- **Structure:**

```
alani-memory/
├── Cargo.toml          # memory crate (persistent and neural memory)
├── src/
│   ├── lib.rs          # memory subsystem interface
│   └── device.rs       # "MemoryDevice" implementations
└── .github/workflows/ci.yml
```

- **Purpose:** Provides a simulated persistent memory and neural storage.
- **Public API:** Traits/structs for storing/retrieving knowledge vectors, and managing NVM (modeled after real NVRAM).
- **CI/CD:** Build/test, include examples of memory operations.

alani-devices

- **Structure:**

```
alani-devices/
├── Cargo.toml          # device I/O crate
├── src/
│   ├── lib.rs          # exports device traits
│   ├── storage.rs      # storage device model
│   └── network.rs      # (if any) network sim
└── .github/workflows/ci.yml
```

- **Purpose:** Abstracts hardware peripherals (e.g. storage, sensors).
- **Public API:** Defines `trait Device` and specific device types. Device drivers in `alani-kernel` will implement these traits.
- **CI/CD:** Build/test, ensure devices properly send/receive via mocks.

alani-security

- **Structure:**

```
alani-security/
├── Cargo.toml          # security utilities crate
├── src/
│   ├── lib.rs          # high-level API
│   ├── auth.rs         # authentication methods
│   ├── crypto.rs       # cryptographic functions
└── .github/workflows/ci.yml
```

- **Purpose:** Cryptography and auth primitives (keys, signatures).
- **Public API:** Functions for encryption, HMAC, auth checks.
- **CI/CD:** Build/test, `cargo-audit` to catch crypto vulnerabilities[6].

alani-audit

- **Structure:**

```
alani-audit/
├── Cargo.toml
├── src/
│   ├── lib.rs          # audit framework
│   └── storage.rs      # tamper-evident log store
└── .github/workflows/ci.yml
```

- **Purpose:** Logging/auditing subsystem (e.g. append-only logs, alerting).
- **Public API:** Interfaces to record events, query logs.
- **CI/CD:** Build/test, include log data integrity checks.

alani-terminal

- **Structure:**

```
alani-terminal/
├── Cargo.toml          # terminal application (binary)
├── src/
│   ├── main.rs         # interactive shell
│   └── commands.rs     # command implementations
└── .github/workflows/ci.yml
```

- **Purpose:** A command-line shell to interact with Alani.
- **Public API:** Exposes commands (e.g. memory inspect, run agent). Possibly none (it's an app).
- **CI/CD:** Build/test, ensure basic shell commands execute.

alani-userspace

- **Structure:**

```
alani-userspace/
├── Cargo.toml          # example user app or library (e.g. a sample AI
agent)
├── src/
│   └── main.rs         # example usage of Alani syscalls and libs
└── .github/workflows/ci.yml
```

- **Purpose:** Example user-level code (could be a library for writing user programs).
- **Public API:** (If library) offers convenient wrappers.
- **CI/CD:** Build/test examples.

Each repo's `Cargo.toml` lists versioned dependencies. We recommend including each repo in a **Cargo workspace** (top-level `Cargo.toml`) so that one `cargo test --all` covers all crates[8][5]. This also makes CI simpler: a single workflow can run tests for all workspace members. For CI we suggest GitHub Actions templates for Rust (e.g. `rust.yml` which sets up Rust versions and runs `cargo build`/`cargo test`)[9].

3. Corpus Creation Plan

We plan a labeled data corpus to train and evaluate Alani's cognitive capabilities. This includes text/numerical corpora and logs. The following outline follows standard annotated-corpus practices[10]:

- **Data Sources:** Identify open datasets (e.g. text from Project Gutenberg, knowledge graphs, sensor logs, command histories). We may also generate synthetic data. Sources could include public APIs (Common Crawl, Kaggle datasets) or internal simulation logs. Each source's license must allow reuse (open data).
- **Formats & Schema:** Store data in structured formats (JSON, CSV, or Parquet). Define a schema document (Doc54) detailing fields (e.g. `"id"`, `"content"`, `"label"`). For example, a JSON entry might be:

  ```
  {
    "id": 123,
    "query": "What is the weather today?",
    "context": "... previous conversation ...",
    "response": "The weather is sunny.",
    "source": "simulator",
    "labels": ["dialogue", "weather"]
  }
  ```

- **Labeling:** Develop annotation guidelines (Doc55) defining the taxonomy of labels. We will conduct annotation campaigns (possibly crowdsourced or expert review) with instructions to label data according to tasks (e.g. intent classification, knowledge linking). Annotations are carried out following best practices: "define classes, provide examples, ensure inter-annotator agreement"[10].

- **Storage & Access:** Store the corpus in a controlled repository (e.g. a secure object store like AWS S3 or a database). Implement access controls (require authentication, limit to approved users). Use versioning/backups. The storage plan (Doc56) will specify format (e.g. one JSONL file per data split) and security (encrypt at rest, IAM policies).
- **Sample Data:** Provide sample files (Doc57) illustrating the data format. Include a small JSON sample in the docs.
- **Generation Scripts:** Write scripts (in Rust or Python) to convert raw data into the corpus format (Doc58). For example, a Rust `serde`-based tool could ingest CSV and output JSON. Include example commands and code comments in scripts.

By addressing the full data lifecycle (collection, processing, annotation, and storage) as recommended[10], the corpus will be robust and reusable.

4. Development Workflow

We adopt a collaborative Git workflow with code reviews and CI. Key elements:

- **Branching Strategy:** Use a *GitHub Flow* style. All work is on branches off `main` (or `master`); feature/fix branches have descriptive names (e.g. `feature/memory-management`)[11]. Each branch is pushed to origin.
- **Pull Requests & Reviews:** Changes are proposed via Pull Requests (PRs). A PR should have a clear title/description, reference related issues, and pass automated checks. We require **at least one approving review** before merging[11]. Protected-branch rules can enforce reviews and green CI.
- **Code Reviews:** Reviewers check for correctness, style, and security. We will maintain a review checklist (Doc49) covering code standards and API stability. Comments and suggestions are done via GitHub's review system.
- **Issue Tracking:** Use GitHub Issues (or similar) for bugs and features. Label issues by type (bug, feature, docs). Link commits/PRs to issues (using `#issue` notation).
- **Testing:** Each crate includes unit tests (`#[cfg(test)]`) and integration tests (`tests/` or `examples/`). Running `cargo test --workspace` verifies all. We also include a continuous fuzzing or property-test step if relevant.
- **Benchmarking:** We use Criterion.rs for microbenchmarks. For example, kernel scheduling and AI inference are benchmarked (Doc44). Criterion "specializes in statistically rigorous analysis" and outputs reports[7]. Benchmarks live in `benches/`, run via `cargo bench`.
- **Static Analysis:** Tools like `rustfmt` (style) and `clippy` (lints) run on every commit. We also use `cargo-audit` (or RustSec tooling) to scan for dependency vulnerabilities[6].
- **Security Audits:** Scheduled reviews by security experts will analyze cryptographic usage, sandboxing, and threat scenarios. We'll update policies (Doc16) as needed.

- **CI/CD:** Every PR triggers the CI pipeline: build all crates, run tests, format check, clippy, and security scans. Successful CI build artifacts will be archived (e.g. for later packaging).

5. Build and Packaging

To produce releases, we automate build and packaging scripts:

- **Building:** Use a top-level script (e.g. Makefile or Rust script) that does:
- `cargo fmt -- --check` (style), `cargo clippy` (lint), and `cargo test --all`.
- `cargo build --release --workspace` (build optimized binaries).

- **Example Packaging Script (build_release.sh):**

```
#!/bin/bash
set -e
# 1. Clean and ensure tests passed
cargo fmt -- --check
cargo clippy -- -D warnings
cargo test --workspace
# 2. Build release binaries
cargo build --workspace --release
# 3. Create a release ZIP
mkdir -p release
cp target/release/alani-kernel release/
cp -r docs/* release/
cp -r corpus/data sample_data/ release/
# 4. Include supporting scripts and readme
cp scripts/*.sh release/
cp README.md LICENSE release/
(cd release && zip -r ../alani_release.zip .)
```

- **Artifacts:** The final `alani_release.zip` contains: all built binaries (in `release/`), all 62 documentation files (`docs/`), corpus sample data (`release/data/`), and utility scripts. We will commit this zip to a GitHub Release, or attach to the project deliverable.
- **Instructions:** Detailed steps (Doc51) will guide a developer to run these scripts, or use Docker/CI pipelines to produce identical builds.

6. Templates and Standards

We provide templates and guidelines to ensure consistency:

- **Documentation Templates:** A Markdown template (Doc59) for new spec documents (cover page, table of contents, section headings, references). This encourages uniform formatting.
- **Code Comments and Examples:** We include code comment guidelines: each public function/struct should have a `///` doc comment; examples in docs. Inlining code examples with doctests is encouraged.
- **Coding Standards:** We adopt the official Rust style guide: 4-space indent, trailing commas on multi-line lists, etc.[2]. Developers must format code with `rustfmt` (default settings). Naming follows Rust API Guidelines[12] (e.g. `snake_case` for functions, `CamelCase` for types). We enforce style with CI.
- **Contribution Guide:** In addition to the simple CONTRIBUTING.md, we supply a detailed doc (Doc61) explaining PR conventions, commit message style, and linking the GitHub Flow guide. A Code of Conduct is included for community standards.
- **API Guidelines:** We follow the [Rust API Guidelines] recommendations[12]. For example, we design library APIs to be consistent and documented. Public crates have a stable semantic versioning policy.
- **Commit Convention:** We encourage conventional commits (e.g. `feat:`, `fix:` prefixes) to generate change logs automatically.

Throughout development, we cite **official Rust sources**. For instance, splitting large projects into workspaces is recommended by the Rust Book[1], and using workspaces to share one `Cargo.lock` is standard practice[5]. CI pipelines will use GitHub's Rust workflow templates[9]. Benchmarks use Criterion for statistical rigor[7]. Security scans use the RustSec database and `cargo-audit`[6].

7. Assumptions

- **Target Platform:** 64-bit x86 (UEFI boot) or RISC-V; details TBD.
- **License:** Open-source (assume Dust Open Source License).
- **Rust Toolchain:** Stable Rust 2021 edition; nightly only for benchmarks if needed.
- **Hardware:** No GPU or network stack required initially. Focus is CPU-based AI.
- **Corp Domain:** Data is assumed to be text/dialogue. We do not address image/audio data.

This comprehensive plan (documents, repos, corpus, workflows, and scripts) equips Rust developers to build Alani from scratch. All references above point to **official Rust sources and best practices** (e.g. Cargo workspaces[5], Rust style guide[2], RustSec tooling[6]) and respected industry guides, ensuring the project follows current standards. The included tables and diagrams (e.g. Figure above, Mermaid graph) clarify architecture and repo layout. Developers can use this report directly to create the 62-spec docs, initialize each repo, write code, and automate builds into the final release ZIP.

Sources: Rust documentation and community resources on project structure[1][5], OS architecture guides[3][4], benchmarking[7], data annotation[10], and security tools[6]. All are cited above.

---

[1] Packages, Crates, and Modules - The Rust Programming Language

https://doc.rust-lang.org/book/ch07-00-managing-growing-projects-with-packages-crates-and-modules.html

[2] Introduction - The Rust Style Guide

https://doc.rust-lang.org/style-guide/

[3] Monolithic kernel - Wikipedia

https://en.wikipedia.org/wiki/Monolithic_kernel

[4] Microkernel - Wikipedia

https://en.wikipedia.org/wiki/Microkernel

[5] Workspaces - The Cargo Book

https://doc.rust-lang.org/cargo/reference/workspaces.html

[6] About RustSec › RustSec Advisory Database

https://rustsec.org/

[7] Benchmarking Rust code using Criterion.rs

https://engineering.deptagency.com/benchmarking-rust-code-using-criterion-rs

[8] Multi crate repository vs multiple repositories - help - The Rust Programming Language Forum

https://users.rust-lang.org/t/multi-crate-repository-vs-multiple-repositories/9444

[9] Building and testing Rust - GitHub Docs

https://docs.github.com/en/actions/tutorials/build-and-test-code/rust

[10] arxiv.org

https://arxiv.org/pdf/2601.13353

[11] GitHub flow - GitHub Docs

https://docs.github.com/en/get-started/using-github/github-flow

[12] About - Rust API Guidelines

https://rust-lang.github.io/api-guidelines/about.html